QA evidence — tmx-orchestrator: Containers-submodule distribution orchestrator

Run-id: 2026-06-16-tmx-orchestrator **Captured:** 2026-06-16 **Conductor:** Mistborn (Darwin arm64, go 1.26.2) · **Target:** nezha.local (ALT Linux x86_64, podman 5.7.1) **Deliverable (operator mandate):** “Make a proper binary using the Containers submodule lib to orchestrate the distribution — for when/if we need it for our testing needs.”

A real consumer binary at `scripts/tmx-orchestrator/` (module `digital.vasic.tmux/orchestrator`) imports the decoupled `digital.vasic.containers` submodule (via `replace => ../../Containers`) and orchestrates container distribution to nezha. The submodule was NOT modified for tmux-specifics (CONST-051); the ONE library change is a generic `Ports` capability (committed upstream `1b9da9b`, §11.4.76 extend-don’t-reimplement).

---

1. `hosts` — real cross-host probe (macOS → nezha over SSH)

```
$ tmx-orchestrator-bin hosts --env Containers/.env
[INFO] registered remote host: nezha (nezha.local)
[DEBUG] ssh exec on nezha: cat /proc/stat && ... /proc/
meminfo ... /proc/loadavg ... nproc ...
NAME   ADDRESS      PORT  REACHABLE  CPU%  MEM%  MEM(MB)     CORES
nezha  nezha.local  22    yes        17.2  9.9   6365/64086  8
```

Real `/proc` resources parsed from nezha over SSH; exit 0.

2. `distribute` — deploy a REAL container on nezha + health-check (the anti-bluff core)

```
$ tmx-orchestrator-bin distribute --image docker.io/library/
nginx:alpine \
      --name tmx-orch-demo --port 80 --publish 18080 --health http
--health-path /
[INFO] batch: scheduled tmx-orch-demo -> nezha (score=0.645)
[INFO] deploying tmx-orch-demo on nezha: podman run -d --name tmx-
orch-demo -p 18080:80/tcp docker.io/library/nginx:alpine
[INFO] distribution complete: 0 local, 1 remote, 0 failed in 730ms
```

```
Distribution summary:
  total=1  local=0  remote=1  failed=0
  CONTAINER      HOST   STATE    ERROR
  tmx-orch-demo  nezha  running

[INFO] health-checking http nezha.local:18080/ (polling until
ready)
Health check (http) nezha.local:18080 -> HEALTHY (19ms)
  status_code: 200
distribute exit=0
```

Independent confirmation on nezha:

```
$ ssh nezha 'podman ps --format "{{.Names}}|{{.Status}}|
{{.Ports}}"'
tmx-orch-demo|Up 1 second|0.0.0.0:18080->80/tcp
$ ssh nezha 'curl -s -o /dev/null -w "HTTP %{http_code}" http://
localhost:18080/'
HTTP 200
```

A genuine nginx container is running on nezha, published on host port 18080, serving HTTP 200.

3. down — teardown, verified clean

```
$ tmx-orchestrator-bin down --name tmx-orch-demo --env
Containers/.env
[INFO] removed container tmx-orch-demo on nezha
teardown complete for container "tmx-orch-demo"
down exit=0
$ ssh nezha 'podman ps -a --format "{{.Names}}" | grep -q tmx-
orch-demo && echo PRESENT || echo REMOVED-CLEAN'
REMOVED-CLEAN
```

4. Library extension proof (Containers `1b9da9b`)

- `ContainerRequirements.Ports []PortMapping` + `-p host:container[/proto]` rendered in `deployRemote`.
- Unit test `TestBuildPublishFlags` PASS 6/6.
- Bluff-audit (Seventh Law): `buildPublishFlags()` forced to `return ""` → `TestBuildPublishFlags` FAIL (`got "" want " -p 1:2/tcp -p 3:4/tcp"`) → restored → PASS; residue scan clean.
- Full `pkg/distribution` + `pkg/scheduler` tests PASS on the integrated upstream tree (`20173e8` + change).
- Containers full unit suite on nezha: all 37 packages `ok`, RC=0.

5. Root-cause fixes during bring-up (systematic-debugging, FACT)

Symptom	Root cause (FACT)	Fix
health check “connection refused” on nezha:80	container run with no `-p`; port only in container netns	added `Ports` to the lib + `--publish` (extend §11.4.76)
deploy fail “8080 Address already in use”	nezha already serves on 8080 (curl → 404)	pick a verified-free host port (18080)
health “connection reset by peer” immediately after deploy	readiness race — first connect before nginx + pasta port-forward accept	poll the health check ~15 s until ready (no false PASS)

All three were diagnosed to FACT root cause before fixing (no guessing, §11.4.6).